Welcome to the Introduction to Nautilus Workshop!

Please complete the check-in below

go.unl.edu/gp-engine-june-sign-in

GP-ENGINE

GP-ENGINE – Migrating AI/ML workflows to Nautilus

Introduction to Containers and Kubernetes

Support provided by NSF OAC#2322218

June 2026

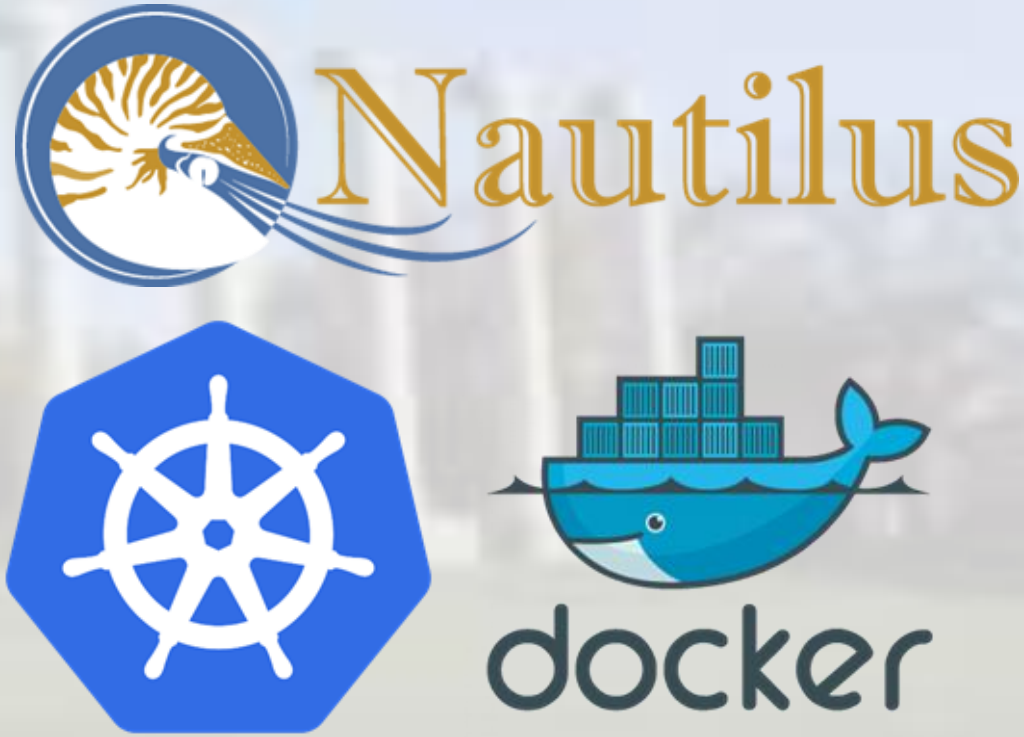
Mathew Keeler- Mizzou HPC
University of Missouri
https://jhurt.mufaculty.umsystem.edu/

Schedule

- 1:15 – 2:00 -> Introduction to NRP and Containers
- 2:00 – 3:15 -> Hands-On with NRP
- 3:15 – 3:45 -> Open Questions

NSF NRP Nautilus HyperCluster

- The NSF Nautilus HyperCluster is a Kubernetes cluster with vast resources that can be utilized for various research purposes:
  - Prototyping research code
  - S3 cloud storage for data and models
  - Accelerated small-scale research compute
  - Scaling research compute for large scale experimentation
- Resources Available:
  - CPU Cores: ~31,400
  - Nodes: 491 across 125 sites
  - NVIDIA GPUs: ~1,500



Nautilus Logo: https://portal.nrp-nautilus.io/
Docker Logo: https://developers.redhat.com/blog/2014/05/15/practical-introduction-to-docker-containers
Kubernetes Logo: https://commons.wikimedia.org/wiki/File:Kubernetes_logo_without_workmark.svg

NSF NRP Nautilus HyperCluster

- Many applications are ready to use and ran by the NRP team for use.
  - Jupyter Hub
  - Coder
  - GitLab
  - BentoPDF
  - LLM Services
    - Open WebUI
    - API
  - Many more: https://nrp.ai/documentation/userdocs/start/resources

Learning Objectives

- Containerization and Docker
- Kubernetes Architecture and Concepts
- Hands-on with Nautilus HyperCluster
- Deploying Pods and Jobs in Kubernetes using Nautilus

GP-ENGINE

Part 1
Software Containerization with Docker

Introduction to Containers and Kubernetes

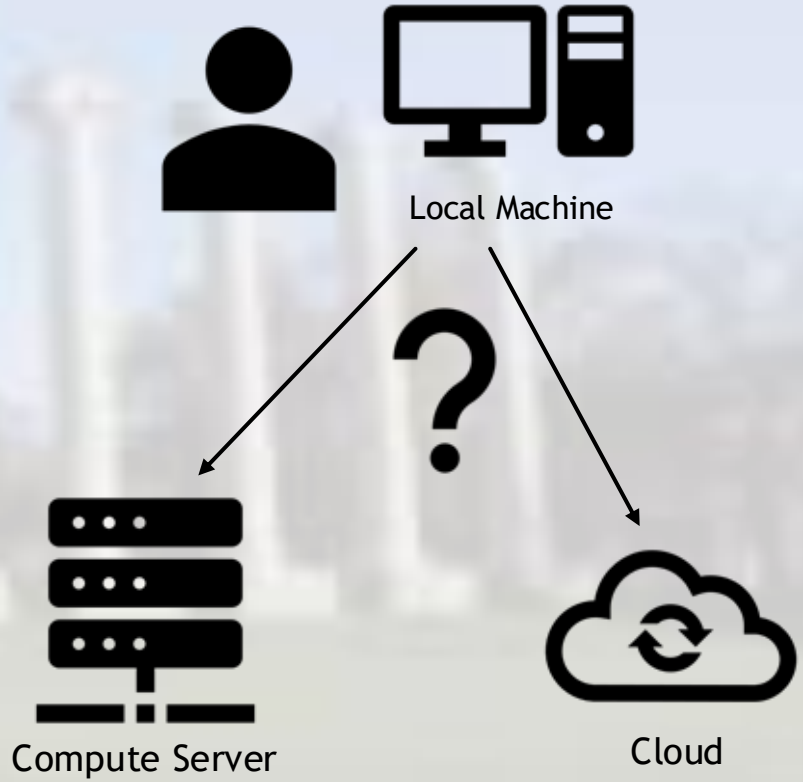
Mathew Keeler- Mizzou HPC
University of Missouri
https://jhurt.mufaculty.umsystem.edu/

Introduction

What is Docker and what problem is Docker trying to solve?

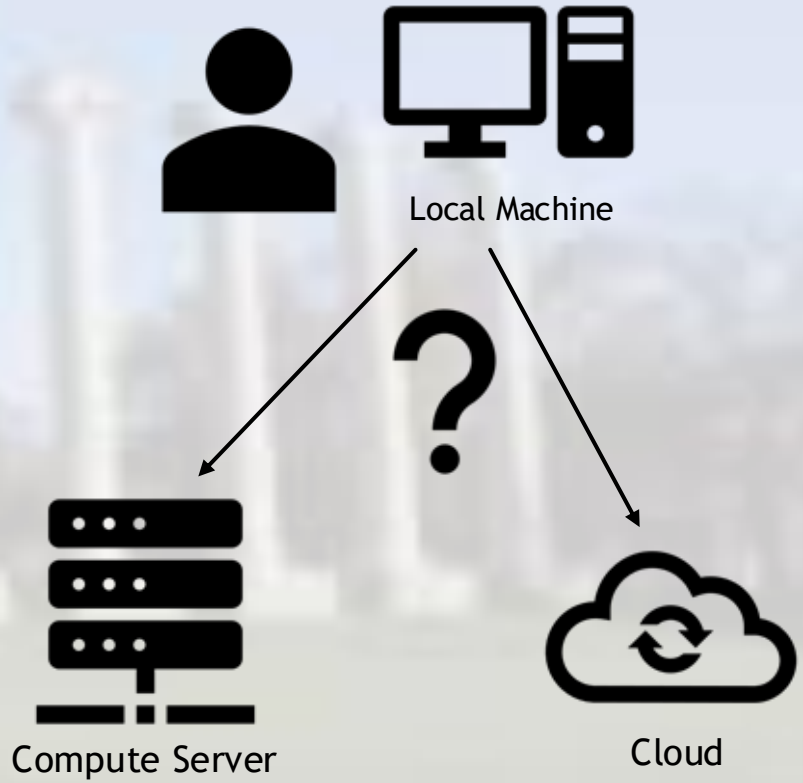
The Problem:
???

- What challenges have you had with using scientific software or software in general?
- Share with your neighbors and try to come up with a list of common challenges or annoyances.



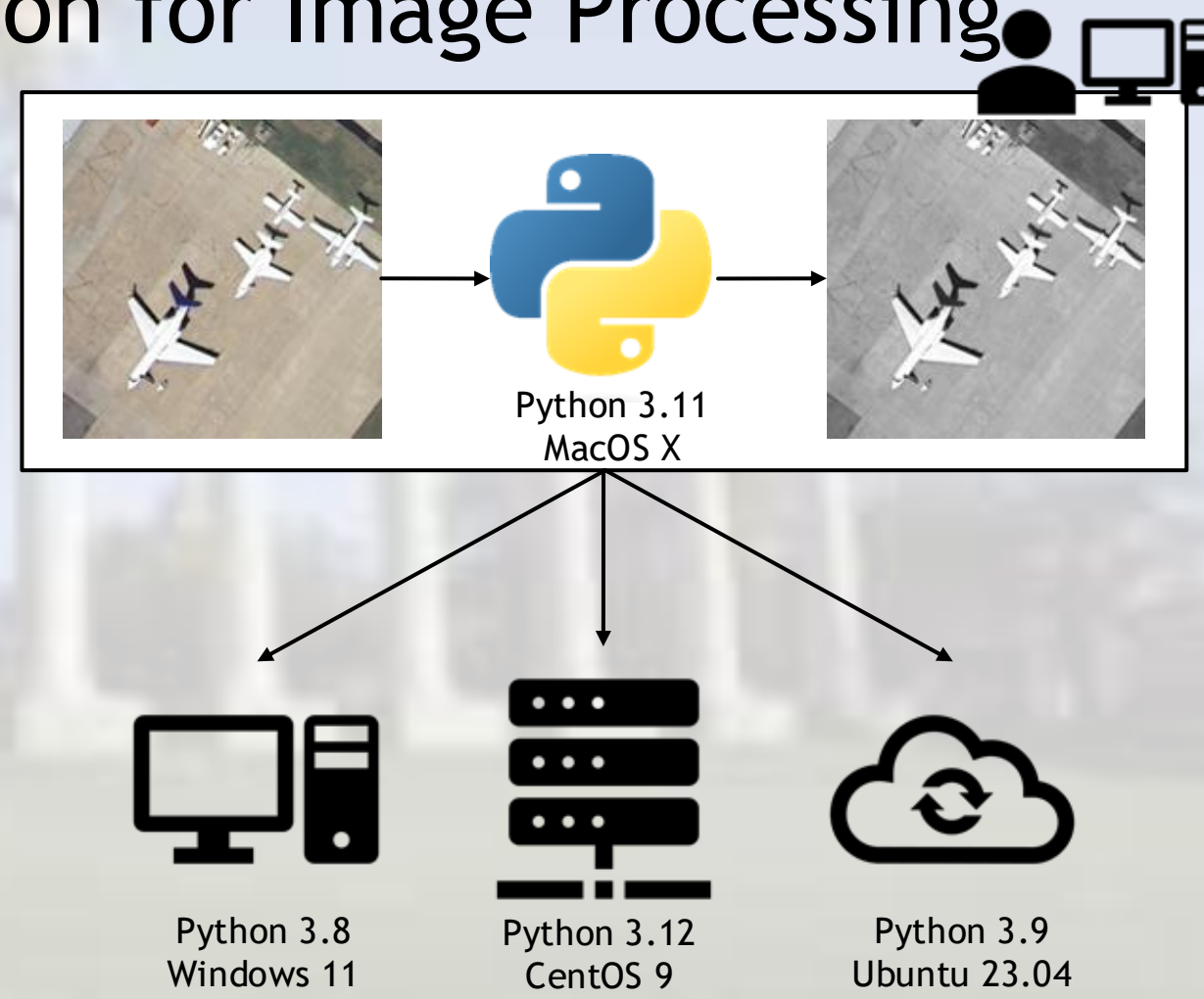
The Problem: Scalability & Reproducibility

- Development often occurs on local development machines
- How do we ensure reliable portability of the software developed on local development machines to other computational environments?
- How do we move code from local development to running on large servers on large swaths of data?



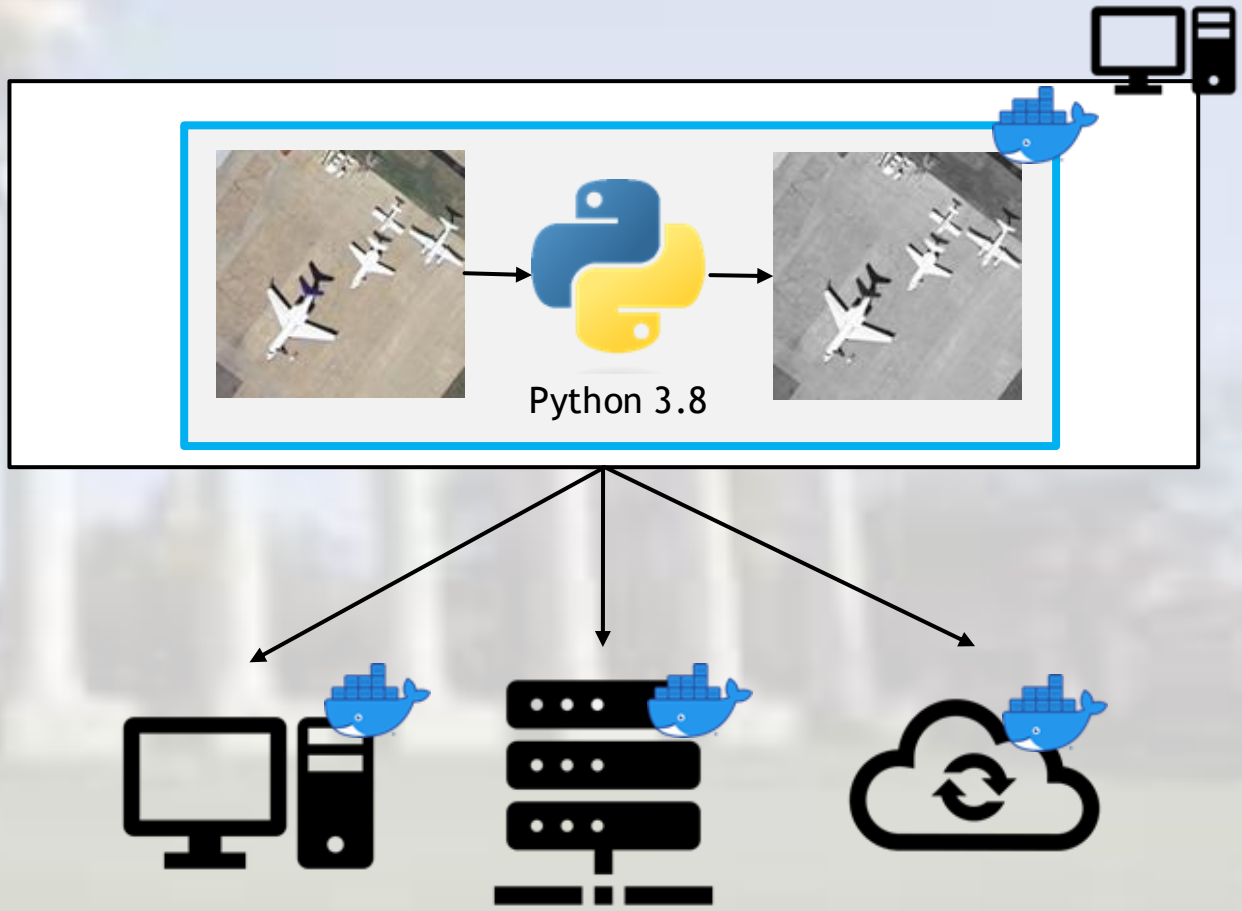
Example: Python Application for Image Processing

- I've built an application on my local machine to enable efficient image processing using Python 3.11
- My application requires certain Python packages **AND** system packages
- The specific versions of Python and the system packages are required
- How do I move my application to co-worker's computer? What about to a compute server? To the cloud?
  - Each of these locations will have their own installed system libraries, python installations, and python packages



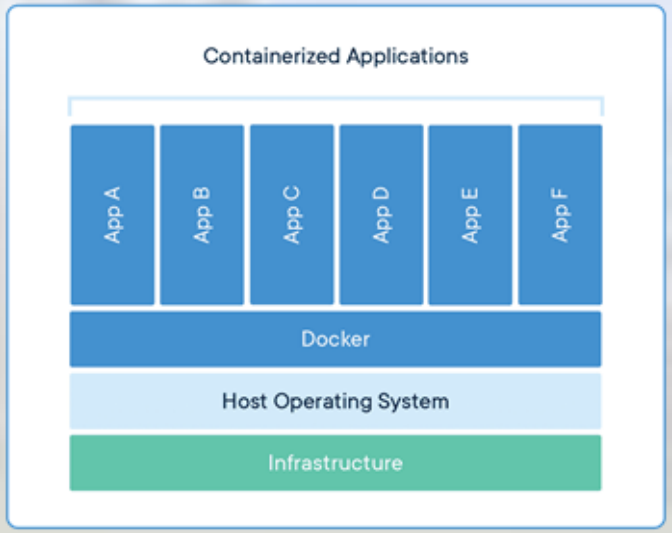
Containerization

- Solution: Package all of our requirements into a software package, called a **container image**, that can be run **anywhere**
- Container images are a single component that contains all your software, dependencies, and code into a single package.



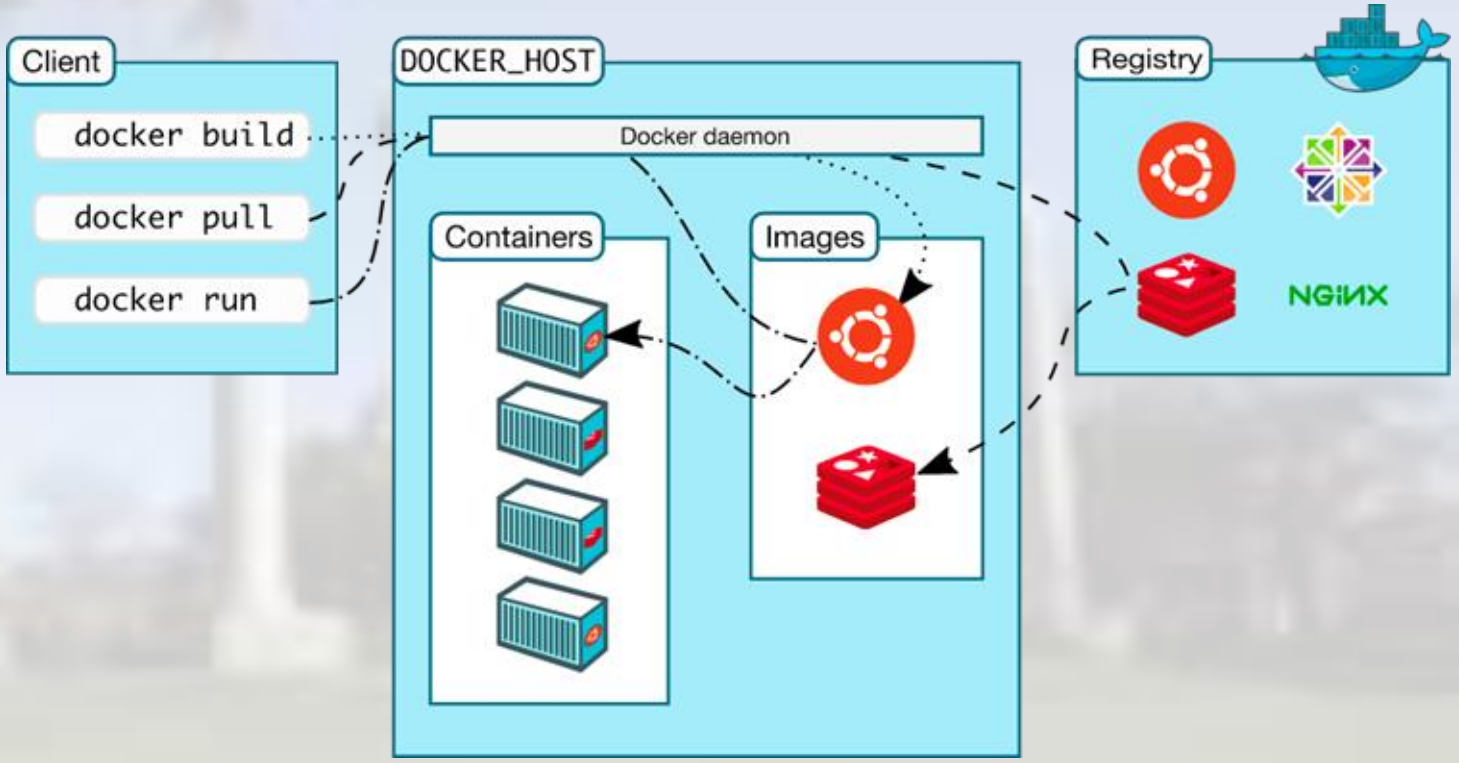
How are containers ran? You may have already used containers!

- Containers and container images only need a single application installed on a computer.
- These applications are referred to as container runtimes, commonly Docker.
- You may have used Apptainer or an Interactive app on Open OnDemand.
  - Apptainer is used to run images on HPC clusters
  - Many Open OnDemand apps are based on images.
- We will discuss Docker here, as its concepts are generalizable to other runtimes including Apptainer



Docker

- **What is Docker?**
  - Docker is a set of tools using virtualization to deliver software in packages called containers.[1]
  - You can think of Docker containers as mini-VMs that contain all the packages, both at the OS and language-specific level, necessary to run your software.
- **Why Docker?**
  - Docker enables predictable and reliable deployment of software.
  - Docker containers are portable!
    - local development computers, compute clusters, internal compute servers, cloud infrastructure, and more!
- *Docker containers enable reliable portability of software to nearly any compute environment*



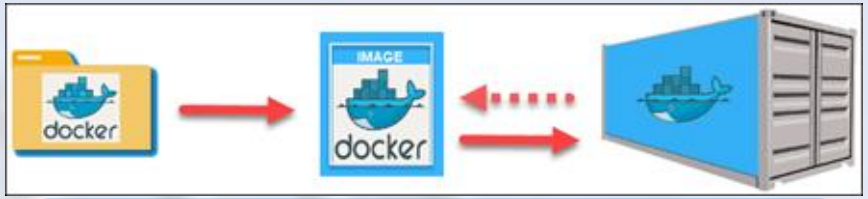
1. https://en.wikipedia.org/wiki/Docker_(software)
2. Image: https://docs.docker.com/get-started/overview/

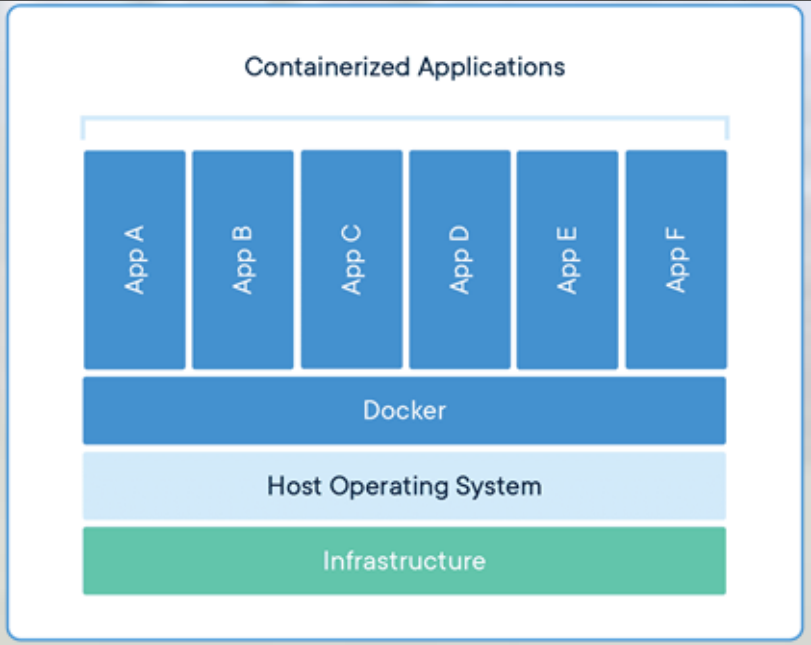
Docker Concepts

Key concepts: containers, container images, Dockerfiles, container registries

Key Container Concepts

- **Image** - Standard unit of software that packages up code and its dependencies, so the application runs reliably from one computing environment to another.
  - Includes everything needed to run an application: code, runtime, system tools, system libraries and settings.
- **Dockerfile** - A list of commands and instructions describing how to build an Image
- **Container** - A container is a standard unit of software that packages up code and all of its dependencies, so the application runs reliably from one computing environment to another.
- **Registry** - a service for storing container images

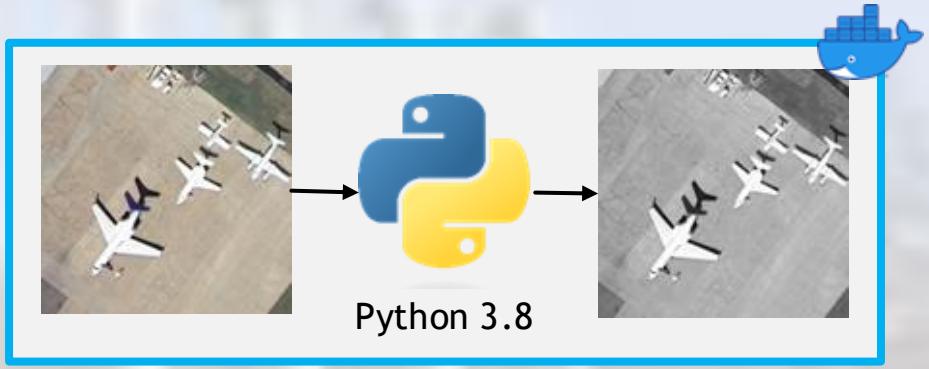




https://www.docker.com/resources/what-container/
https://phoenixnap.com/kb/docker-image-vs-container

Containers vs Container Images

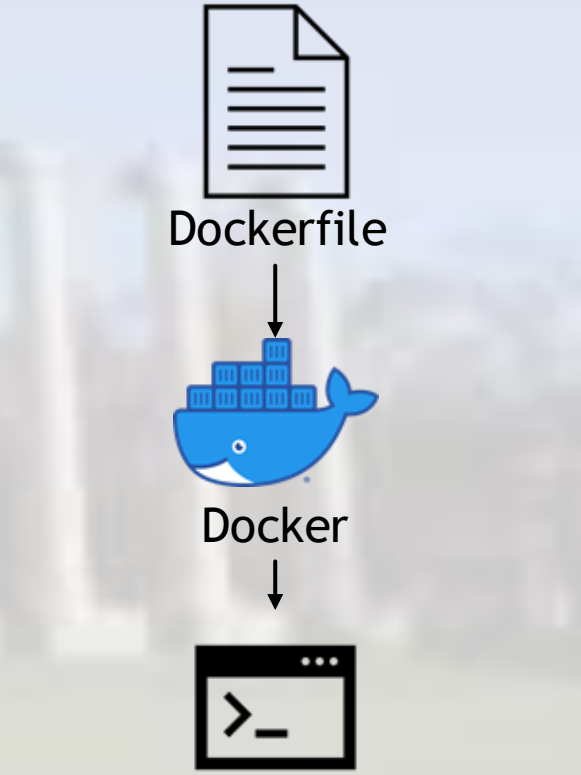
- Container images are software packages that include all necessary software to run the application/library
- Containers are an instance of container images
  - Images become containers at runtime
  - Analogous to relationship between a script and the process or a class and an object



Container Image

Dockerfiles

- Recall: Containers and container images enable reliable portability of software
- We need a way to build container images in a distributed and standardized way, so that anyone with a container runtime can build and run our software package (container image)
- Dockerfiles are the template, or recipe, for how a container image will be built
- Dockerfiles use a custom format and set of commands to describe how a container image will be built



Using Community Published Container Images

- Container images are extensible!
  - Containers can be built from other images
- We can then build hierarchical docker containers, where each container has a specific set of dependencies and packages installed
- Base images enable rapid and reproducible building of even complex docker containers by leveraging the open source, published docker images

Sample Dockerfile

```
ARG PYVERSION=3.8
FROM python:${PYVERSION}

RUN mkdir -p /workspace
WORKDIR /workspace

COPY /requirements.txt /workspace
RUN pip install -r ./requirements.txt

COPY /*.py /workspace/
CMD /bin/bash
```

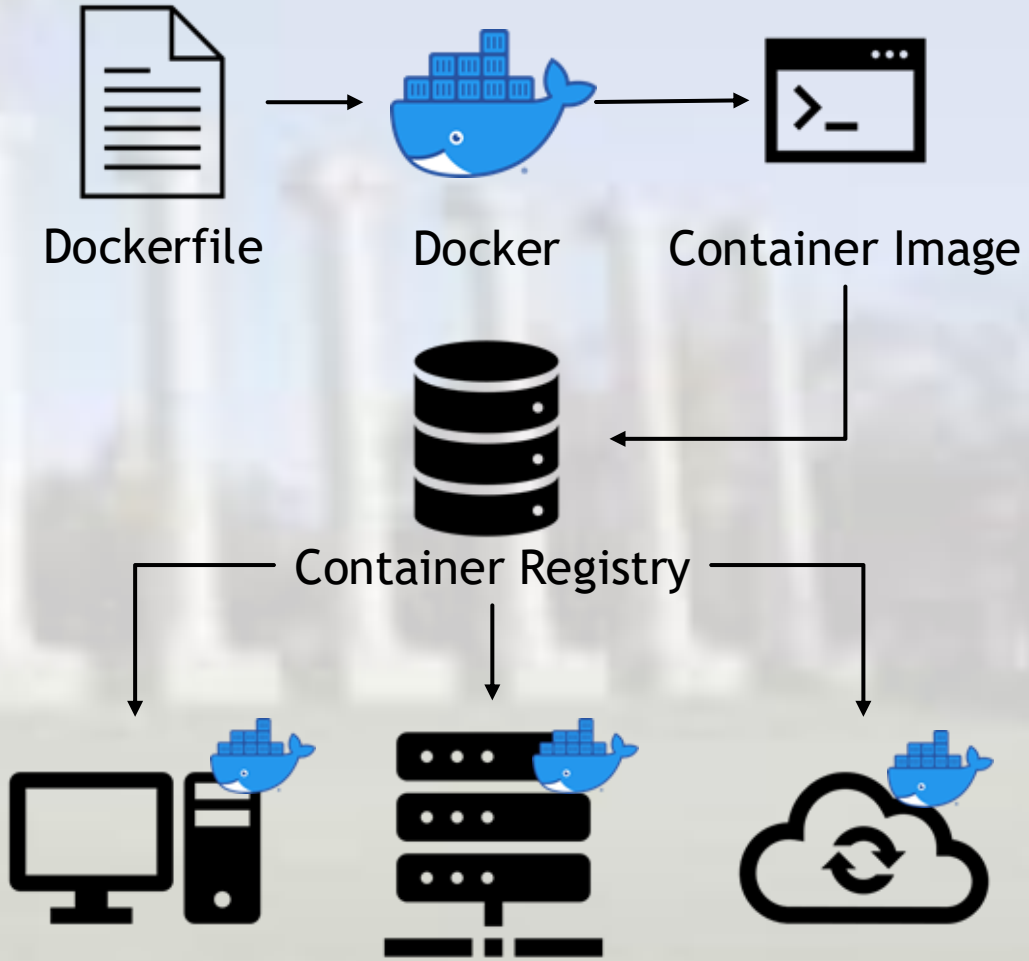
- Create a build argument for the python version that defaults to 3.8
- Start FROM an existing image in a Docker *registry*. In this case, python
- Create a directory named /workspace and set it as the working directory
- Copy a file named “requirements.txt” from the build directory to /workspace in the container
- Use pip to install the requirements defined in requirements.txt
- Copy all python files in the build directory to the /workspace directory in the container
- Set the default command to run when the container starts to /bin/bash

Docker Image Registries

- Container Registries are web-enabled storage locations for container images
  - Similar to Google Drive for documents and spreadsheets
- Each image published on a registry contains a name and a tag:
  - python:3.8 ▫ Python is the name of image and 3.8 is the tag
- If a URL is not specified, the default registry used is Docker Hub
  - https://hub.docker.com/
- Other Container Registries
  - Docker can work with third party container registries when given full URL to the image
  - Example: nvcr.io/nvidia/pytorch:22.08-py3
- Security and Visibility
  - Container images published on registries can be public or private

Revisiting our Example

- We can publish our container image to a registry, and then pull down our container image in each computing environment in which we want to use it
- We can utilize public registries, like Docker Hub, or private ones, to control who we allow to pull down our container



GP-ENGINE

Part 2
Container management with Kubernetes

Introduction to Containers and Kubernetes

Mathew Keeler- Mizzou HPC
University of Missouri
https://jhurt.mufaculty.umsystem.edu/

Subsection Outline

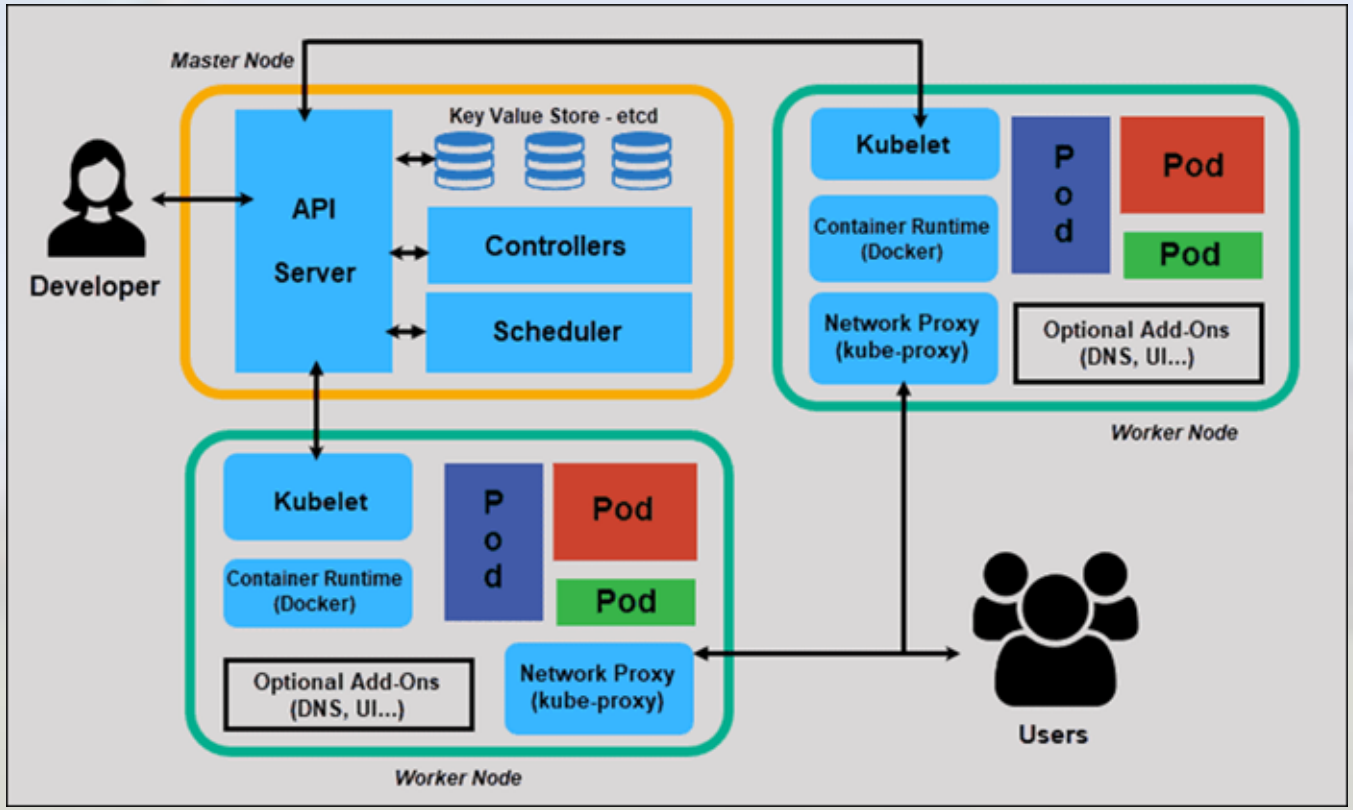
- Introduction to Kubernetes
- Kubernetes concepts
- Kubernetes usage
- Kubernetes hands on

Introduction to Kubernetes

What is Kubernetes and what is it used for

Kubernetes

- Kubernetes, also known as K8s, is an open-source system for automating deployment, scaling, and management of containerized applications.[1]
- Kubernetes enables both simple and complex container orchestration
- Kubernetes cluster has two main components
  - Master node
  - Worker node



1. https://kubernetes.io/
2. Image: https://phoenixnap.com/kb/understanding-kubernetes-architecture-diagrams
3. Logo: https://commons.wikimedia.org/wiki/File:Kubernetes_logo_without_workmark.svg

Kubernetes

Master Node

- The master node handles the control plane, secrets, schedulers, networking, and API server.
- The API Server is what various tools use to communicate to the cluster and components within the cluster.
- This means workflows can be submitted from any computer with a command line or graphical application.

Worker Node

- The worker nodes are what run the workflows sent to the master node through the API Server.
- Worker nodes can run multiple Pods and Jobs at once if resources are available on the worker node.
- Worker nodes also help coordinate network traffic across different Pods and traffic coming from outside the cluster.

1. https://kubernetes.io/
2. Image: https://phoenixnap.com/kb/understanding-kubernetes-architecture-diagrams
3. Logo: https://commons.wikimedia.org/wiki/File:Kubernetes_logo_without_workmark.svg

Kubernetes concepts

Node, pod, persistent volume, job, deployment, service

Key Kubernetes Concepts
Persistent Volume Claim (PVC)

- To maintain the data generated, a persistent volume claim (storage) is needed
- A **persistent volume claim** is a request for storage by a user. [1]
- There exists different classes of persistent volumes such as:
  - CephFS
  - Rados Block Device (RBD)
  - LinStor
- There are different access modes:
  - ReadWriteOncePod - Only one pod at a time can read and write to the Volume
  - ReadWriteOnce - Only one node at a time and its pods can read and write to the Volume
  - ReadOnlyMany – Can be read by multiple nodes and their pods at the same time
  - ReadWriteMany - Can be read from and written to by multiple nodes and their pods at the same time

1. https://kubernetes.io/

Key Kubernetes Concepts

Pod

- Pods are the basic scheduling unit of K8s.
- Pods typically consist of one container running inside. Each Pod has a unique IP address to enable databases or applications
- Pods can run custom scripts (initcontainer) at runtime to initialize the Pod
- Pods generally have limitations on allocated resources and max runtime
- Pods are stateless, meaning all data uploaded or generated by the pod is deleted when the Pod terminates, unless the data is stored in a Persistent Volume Claim.

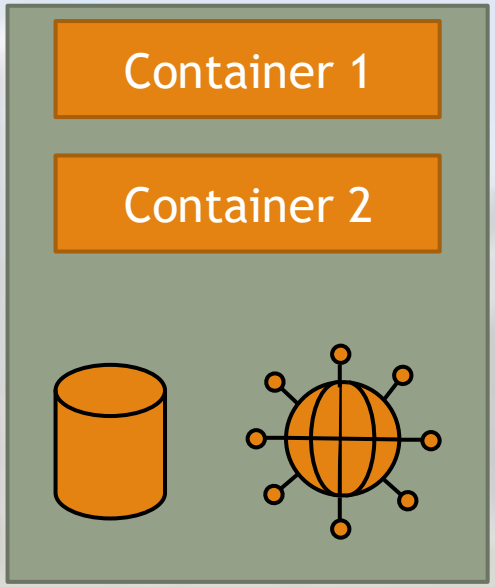


Image: https://aws.plainenglish.io/kubernetes-deep-dive-job-and-cronjob-5ffed1c5fa4e

Key Kubernetes Concepts

Jobs

- A Job creates Pods and will continue to retry execution of the Pods until a specified number of them successfully finish.[1]
- A job has access to a large number of resources and can run for extended periods of time
- A Job may consist of one pod or multiple Pods working in parallel
- Deleting a Job will automatically delete its corresponding Pod
- A Job can create a new Pod(s) if any of its Pod(s) is deleted or failed for any reason.
- Since Jobs create Pods to process data, Jobs are stateless

1. https://kubernetes.io/

Kubernetes usage

Basics of YAML language, kubectl installation and usage

Yet Another Markup Language (YAML)

XML	JSON	YAML

<Servers>
<Server>
<name>Server1</name>
<owner>John</owner>
<created>123456</created>
<status>active</status>
</Server>
</Servers>
	{
	Servers: [
	{
	name: Server1,
	owner: John,
	created: 123456,
	status: active
	}
	]
	}
		Servers:
		- name: Server1
		owner: John
		created: 123456
		status: active

- YAML is a key-value pair file format, similar to JSON and XML
- Kubernetes operations are performed using **YAML** files, known as a Spec file
  - Creating Persistent Storage
  - Creating Pods
  - Creating Jobs
  - Deploying services

Interfacing with Kubernetes: KubeCTL

- With a published Docker image and prepared YAML Spec file, KubeCTL enables interaction with Kubernetes:

kubectl [command] [TYPE] [NAME] [flags]

where:

- command: Specifies the operation that you want to perform on one or more resources, for example create, get, describe, delete
- TYPE: Specifies the resource type, such as pod or job
- NAME: Specifies the name of the resource, or the path to a Spec file
- flags: Specifies optional flags, such as --server to specify the address and port of the API server

https://kubernetes.io/docs/reference/kubectl/

Interfacing with Kubernetes: KubeCTL cheat sheet

- To create pod

```
kubectl create -f pod.yaml
kubectl apply -f pod.yaml
```

- To create job

```
kubectl create -f job.yaml
kubectl apply -f job.yaml
```

Interfacing with Kubernetes: KubeCTL cheat sheet

- To check pod status

```
kubectl get pods
kubectl describe pod pod-name
```

- To check job status

```
kubectl get jobs
kubectl describe job job-name
```

- To debug pod

```
kubectl logs pod-name
```

Interfacing with Kubernetes: KubeCTL cheat sheet

- Access Pod interactively

```
kubectl exec -it pod-name -- /bin/bash
```

- Copy data from Nautilus to local machine

```
kubectl cp pod-name:path/to/data local/path/
```

- Copy data to Nautilus from local machine

```
kubectl cp local/path/ pod-name:path/to/data
```

- Exit interactive Pod mode

Press ctrl+D

Interfacing with Kubernetes: KubeCTL cheat sheet

- To delete pod

kubectl delete –f pod.yaml

Kubectl delete pod-name

- To delete job

kubectl delete –f job.yaml

Kubectl delete job-name

Interfacing with Kubernetes: KubeCTL cheat sheet

- To create persistent volume

  kubectl create –f pvc.yaml

  Kubectl apply –f pvc.yaml

- To increase the size of persistent volume

  kubectl apply –f pvc.yaml

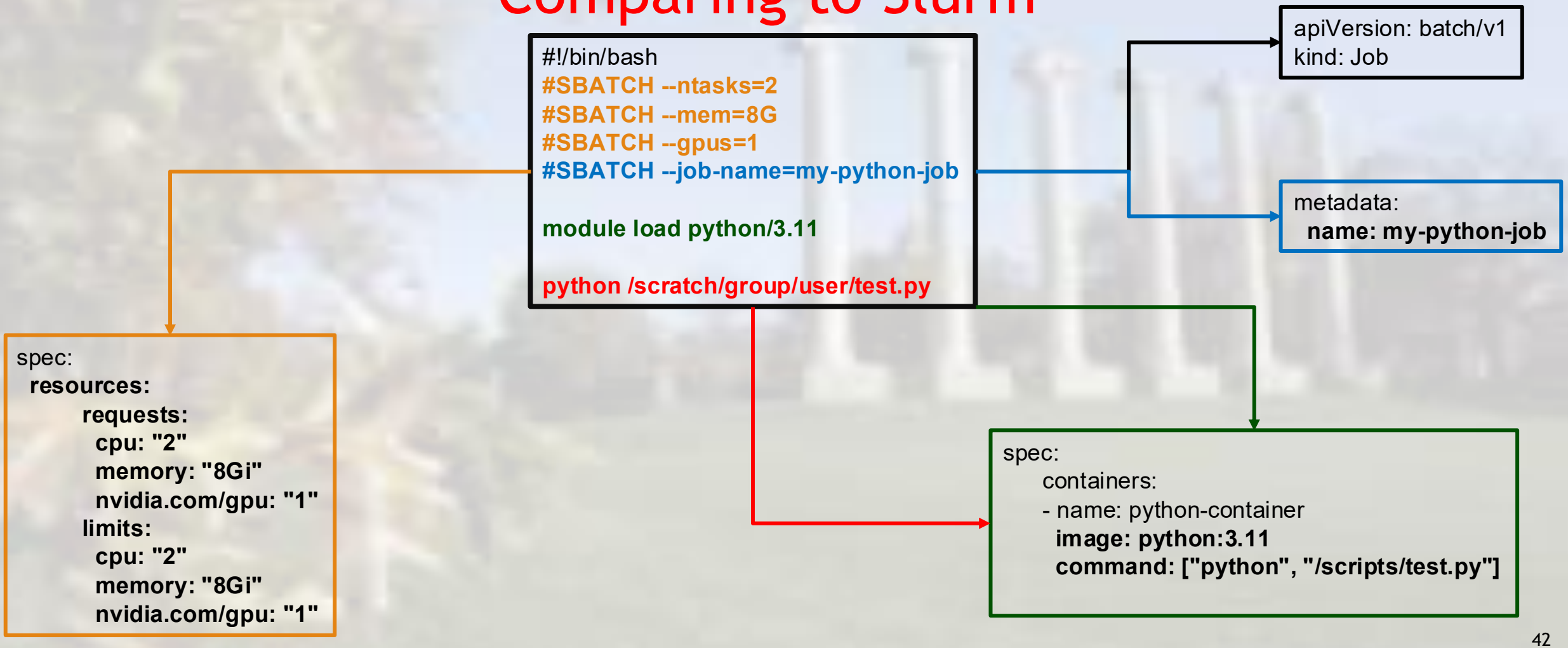
- To delete persistent volume

  kubectl delete –f pvc.yaml

  kubectl delete pvc-name

Key Kubernetes Concepts

Comparing to Slurm



GP-ENGINE

Part 3
National Research Platform
Nautilus Research Cluster

Introduction to Containers and Kubernetes

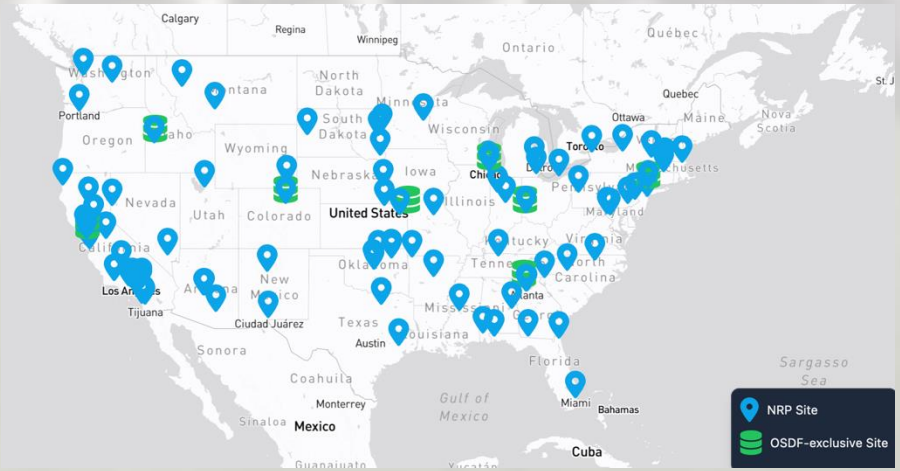
Mathew Keeler- Mizzou HPC
University of Missouri
https://jhurt.mufaculty.umsystem.edu/

A quick note on Kubernetes Clusters, the NRP, Commercial Clouds, and other K8s Clusters

- All commercial cloud providers support Containers and Kubernetes
- The concepts and examples in this tutorial may require minor modifications to adapt to other environments
- We are using the US National Research Platform solely for demonstration and tutorial purposes
- All Container and Kubernetes concepts are portable to commercial clouds or other research Kubernetes platforms

NSF NRP Nautilus HyperCluster

- The NSF Nautilus HyperCluster is a Kubernetes cluster with vast resources that can be utilized for various research purposes:
  - Prototyping research code
  - S3 cloud storage for data and models
  - Accelerated small-scale research compute
  - Scaling research compute for large scale experimentation
- Resources Available:
  - CPU Cores: ~31,400
  - Nodes: 491 across 125 sites
  - NVIDIA GPUs: ~1,500



Nautilus Logo: https://portal.nrp-nautilus.io/
Docker Logo: https://developers.redhat.com/blog/2014/05/15/practical-introduction-to-docker-containers
Kubernetes Logo: https://commons.wikimedia.org/wiki/File:Kubernetes_logo_without_workmark.svg

NSF NRP Nautilus HyperCluster

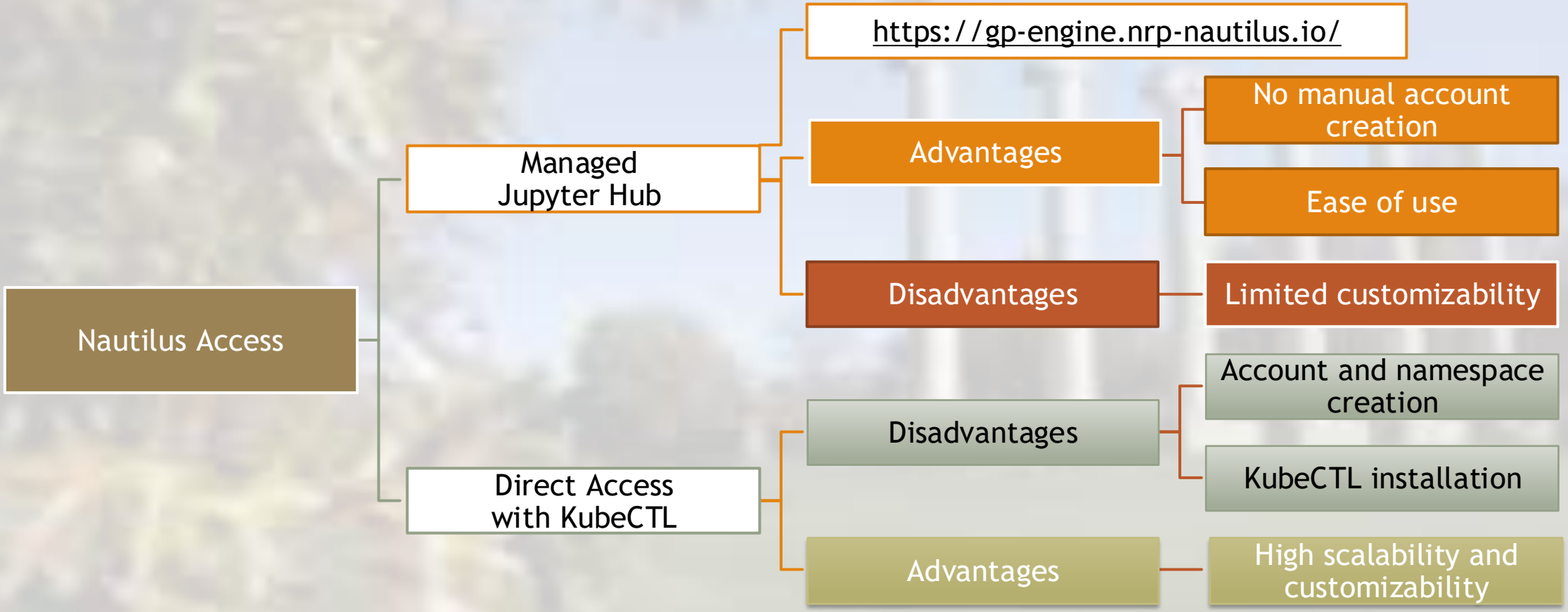
- Storage Classes:
  - Rados Block Device
    - General Purpose Storage for any file.
  - CephFS
    - General Purpose Storage for medium to large files
  - Linstor
    - Low Latency Storage
  - GP-STOR (Coming Soon)
    - Great Plains specific flash storage at UNL, USD, and Mizzou

Nautilus Logo: https://portal.nrp-nautilus.io/
Docker Logo: https://developers.redhat.com/blog/2014/05/15/practical-introduction-to-docker-containers
Kubernetes Logo: https://commons.wikimedia.org/wiki/File:Kubernetes_logo_without_workmark.svg

NSF NRP Nautilus HyperCluster

- Many applications are ready to use and ran by the NRP team for use.
  - Jupyter Hub
  - Coder
  - GitLab
  - BentoPDF
  - LLM Services
    - Open WebUI
    - API
  - Many more: https://nrp.ai/documentation/userdocs/start/resources

Access to Nautilus



Hands on Kubernetes

Life cycle of pods and jobs

Hands on Kubernetes

- Demonstration of pod life cycle (without persistent volume)
  - Creation, interactive access, simple operation, closing, and deletion
- Creation of persistent volume
- Demonstration of pod life cycle (with persistent volume)
  - Creation, interactive access, simple operation, closing, and deletion
- Pod monitoring and debuging
- Demonstration of job life cycle